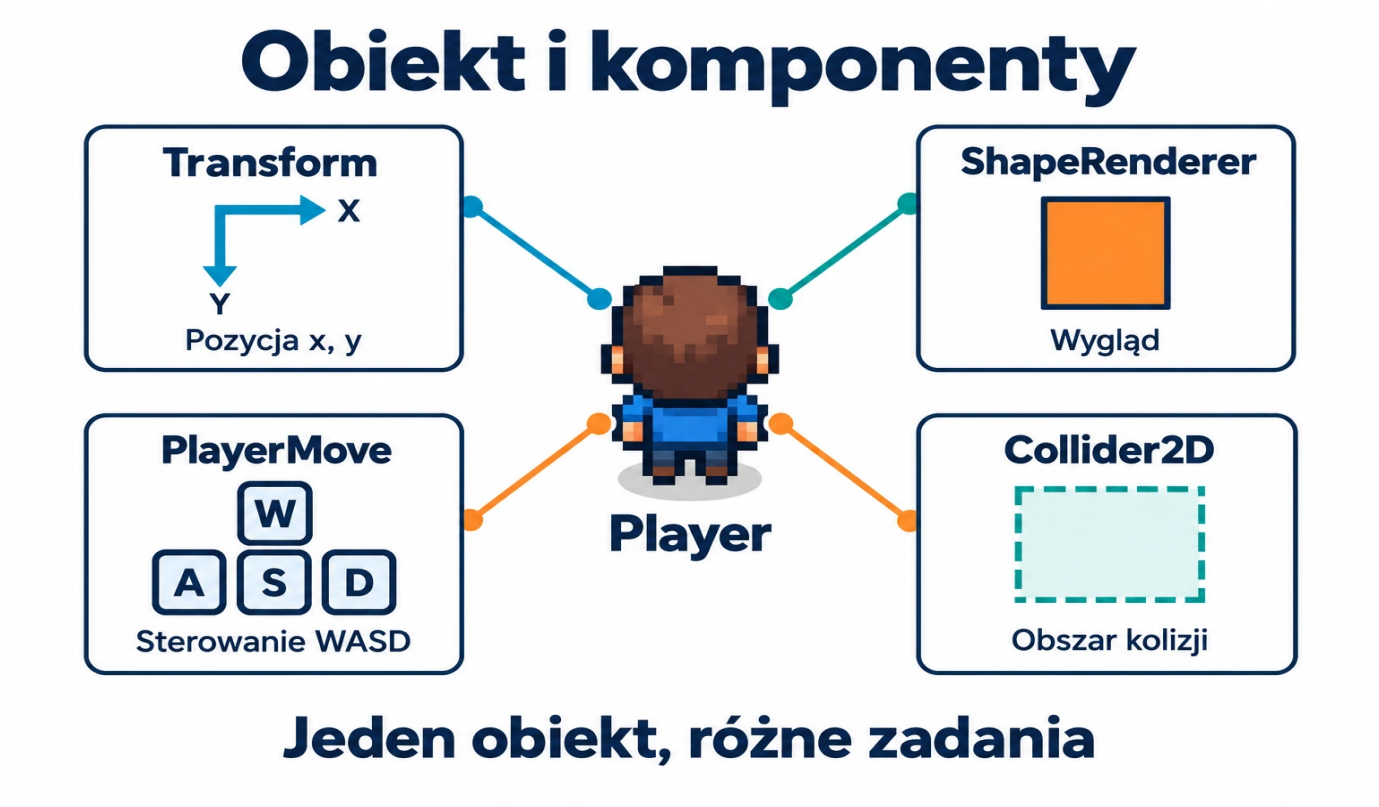
Pierwsza gra w Javie



Pracujesz w Java Lab, w ścieżce Game Dev albo we własnym projekcie Playground. Kod sceny zapisujesz w **GameMain.java**, a własne komponenty w osobnych plikach Java. RUN uruchamia scenę; nie dopisujesz wywołania GameLab.run().

Po tym kursie potrafisz

1. Odczytać pozycję i rozmiar obiektu.
2. Oddzielić tworzenie sceny od aktualizacji w czasie.
3. Złożyć gracza z komponentów i napisać sterowanie WASD.

Wymagane podstawy: zmienne, klasy, metoda i instrukcja if. Każdy pełny plik zawiera import engine.*; edytor Java Lab udostępnia ten sam silnik. Przewiduj wynik przed RUN i porównuj go z zachowaniem gry.

Pozycja to środek obiektu



Bez przesuniętej kamery punkt (0, 0) jest w lewym górnym rogu planszy. X rośnie w prawo, Y w dół. Pozycja obiektu oznacza jego **środek**, a szerokość i wysokość opisują rozmiar grafiki.

```
GameObject player = createObject("Player");
player.setPosition(160, 100);
player.addComponent(new ShapeRenderer(32, 32, "#008f95"));
```

Przewidź: ruch o 40 px w prawo z (160, 100) daje (200, 100). Ruch o 30 px w górę daje (160, 70). Gracz w (100, 100) o wymiarach 32 × 32 px ma po 16 px grafiki z każdej strony środka.

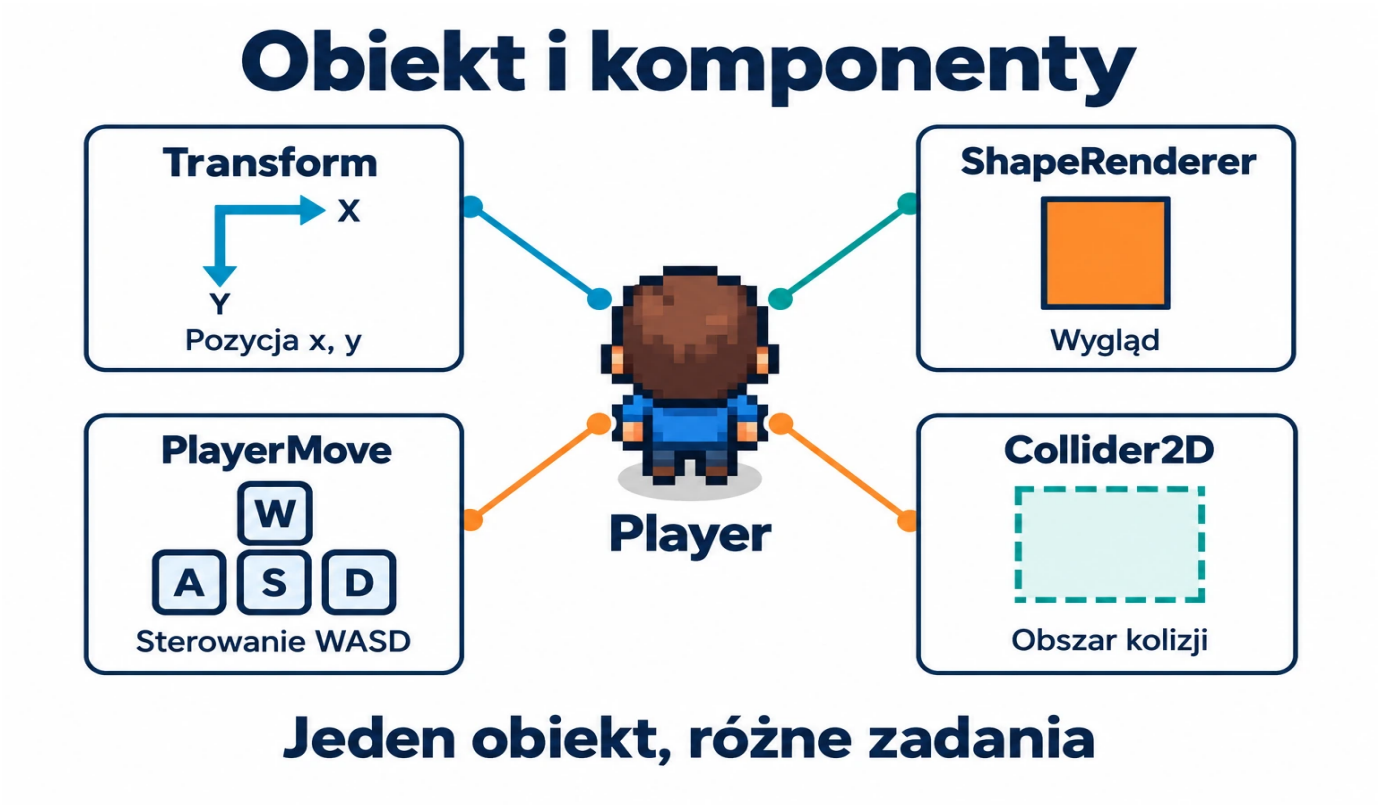
Twórz raz, aktualizuj wiele razy




```
import engine.*;
public class GameMain extends Game {
    @Override public void onCreate() {
        // Utwórz obiekty i dodaj komponenty.
    }
    @Override public void onUpdate(double delta) {
        // Aktualizuj stan gry.
    }
}
```

onCreate() buduje scenę. **onUpdate(double delta)** wykonuje się wielokrotnie. Delta to czas w sekundach od poprzedniej aktualizacji; 60 FPS jest przykładem, a nie gwarancją. Nie twórz kolejnego gracza w każdej klatce.

Obiekt ma stan i zachowania



GameObject reprezentuje Player. Jego **Transform** już istnieje i przechowuje pozycję, obrót oraz skalę. Pozostałe komponenty dodajesz przez addComponent(new ...).

ShapeRenderer rysuje kształt. **Sprite** rysuje teksturę. **CharacterController2D** wykonuje ruch. **PlayerMove** z następnej strony odczytuje klawiaturę. **Collider2D** określa obszar kolizji.

Brak renderera nie usuwa obiektu, ale może sprawić, że go nie widzisz. Sam renderer nie dodaje sterowania. W Javie pobierasz komponent przez getComponent(CharacterController2D.class).

GameMain.java: zbuduj gracza

W Playground umieść poniższy kod w GameMain.java. Dodaj także plik PlayerMove.java z kolejnej strony. Każda publiczna klasa ma własny plik o tej samej nazwie.

```
import engine.*;

public class GameMain extends Game {
    @Override public void onCreate() {
        GameObject player = createObject("Player");
        player.setPosition(160, 100);
        player.addComponent(
            new ShapeRenderer(32, 32, "#008f95")
        );
        player.addComponent(new CharacterController2D());
        player.addComponent(new PlayerMove());
    }
    @Override public void onUpdate(double delta) {}
}
```

Sprawdź wynik

Widzisz jeden kwadrat o szerokości 32 px i wysokości 32 px. Jego środek znajduje się w (160, 100). Kliknij planszę przed użyciem WASD, aby otrzymała fokus klawiatury.

Kolejność: utwórz obiekt → ustaw pozycję → dodaj wygląd → dodaj kontroler → dodaj własne sterowanie. Oddzielne instrukcje ułatwiają znalezienie błędu.

PlayerMove.java: sterowanie WASD

```
import engine.*;

public class PlayerMove extends Component {
    @Override public void onUpdate(double delta) {
        CharacterController2D controller =
            getComponent(CharacterController2D.class);
        InputManager input = getGame().input;
        double x = 0;
        double y = 0;
        if (input.isKeyDown("a")) x -= 1;
        if (input.isKeyDown("d")) x += 1;
        if (input.isKeyDown("w")) y -= 1;
        if (input.isKeyDown("s")) y += 1;
        controller.move(x, y, 120);
    }
}
```

A i D zmieniają kierunek osi X, a W i S osi Y. Bez wciśniętego klawisza kierunek wynosi (0, 0). Kontroler normalizuje ruch po skosie, aby nie był szybszy.

120 oznacza prędkość w px/s. Kontroler sam uwzględnia czas. Nie przekazuj 120 * delta jako prędkości: pomnożyłbyś czas drugi raz. Własny komponent korzysta z getGame().input, a scena GameMain może używać bezpośrednio input.

Zmień i sprawdź

Ustaw start w (320, 180), prędkość 180 px/s i inny kolor. Usuń na chwilę PlayerMove. Wyjaśnij, dlaczego kwadrat pozostaje widoczny, ale przestaje reagować na klawisze.

Delta: ruch niezależny od FPS

Jeśli sam aktualizujesz pozycję bez kontrolera, obliczasz drogę jako prędkość × czas. Poniższy komponent przesuwa obiekt w prawo z prędkością 90 px/s.

```
import engine.*;
public class AutoMove extends Component {
    @Override public void onUpdate(double delta) {
        transform.x += 90 * delta;
    }
}
```

Przy delta = 0,02 przesunięcie wynosi 1,8 px. Przy delta = 0,01 wynosi 0,9 px. Po tej samej sekundzie łączna droga ma być podobna. Nie łącz AutoMove i PlayerMove, jeżeli oba sterują tą samą pozycją.

Mini-projekt: gracz i znacznik

Player: pozycja (80, 80), szerokość 32 px, wysokość 32 px, WASD i prędkość 120 px/s. Marker: pozycja (320, 180), szerokość 24 px, wysokość 24 px, kolor pomarańczowy, bez sterowania.

```
GameObject marker = createObject("Marker");
marker.setPosition(320, 180);
marker.addComponent(new ShapeRenderer(24, 24, "#ff8c28"));
```

Sprawdź rozumienie i typowe błędy

Kryteria mini-projektu

1. Powstają dokładnie dwa obiekty, oba w onCreate().
2. Player ma rozmiar 32 × 32 px; Marker 24 × 24 px.
3. Tylko Player ma kontroler i PlayerMove.
4. Po puszczeniu klawiszy ruch ustaje.
5. Ruch w górę zmniejsza Y.

Typowe błędy

Nieznana klasa: sprawdź nazwę pliku, wielkość liter i import engine.*.
NullPointerException przy move: dodaj CharacterController2D przed sterowaniem.
Obiektów stale przybywa: przenieś createObject() z onUpdate() do onCreate().
Brak ruchu: kliknij planszę, sprawdź klawisze i obecność PlayerMove.

Pytania i odpowiedzi

Czy Transform trzeba dodać ręcznie? **Nie.**
Czy szerokość 32 oznacza pozycję X = 32? **Nie: rozmiar i pozycja to inne wielkości.**
Czy renderer zapewnia sterowanie? **Nie.**
Ile wynosi 90 × 0,02? **1,8 px.**
Co wywołuje RUN? **Silnik tworzy nową scenę i uruchamia jej cykl życia.**

Dalszy krok: kamera i HUD, potem animacje. Fragmenty kodu dodawaj w opisanej metodzie; pełne klasy zapisuj w osobnych plikach Java.